Accelerating Format-Conversion, Tensor-Transpose and Tensor-Dot in DIAQ

Efficient Tensor Computations for Scientific Applications and Deep Learning(CSC591-026) Course Project

Srikar Chundury
North Carolina State University
Raleigh, USA
schundu3@ncsu.edu

Abstract

This paper outlines the ongoing enhancements made to the DIAQ numerical library subsequent to the conclusion of the parallel systems course project, overseen by Dr. Jiajia Li. Our focus this semester was on refining the library's functionality and performance in various aspects.

Specifically, our efforts were directed towards optimizing key tensor operations, such as tensordot and transpose, which were identified as critical bottlenecks in our previous work. We analyzed existing algorithms to identify areas for improvement. Subsequently, a migration from SSE4.2 to AVX2 was attempted, albeit resulting in a temporary performance dip. In-depth analyses utilizing tools like likwid revealed that byte-aligning float arrays improved AVX2 vectorization, particularly in specific scenarios like the 14-qubit case (see Figure 8).

Moreover, we addressed a significant bottleneck by developing a CUDA kernel to facilitate seamless format conversion between NumPy arrays and DIAQ objects. Despite these advancements, the runtime efficiency of the HAM benchmark quantum circuit remained largely unchanged, possibly due to inherent matrix sparsity.

Additionally, a CUDA kernel for spMV for DIAQ was implemented, targeting statevector simulations. We also focused on bolstering the library's infrastructure by improving building, linking, and testing frameworks using tools like cmake, make, and bash scripts.

Our ongoing work signifies a continuous evolution of the DIAQ library, promising significant advancements in the realm of digital quantum simulations encompassing state vectors and tensor networks.

Keywords: Quantum Computing, High Performance Computing, Tensor Networks, Efficient Tensor Computations

1 Introduction

The race between Digital Quantum Simulations and Analog Quantum Simulations to demonstrate quantum supremacy [3], the ability of a programmable quantum computer to solve problems beyond the reach of classical computers within feasible time frames, defines a pivotal juncture in quantum computing. Efficient Digital Quantum Simulations leveraging High-Performance Computing (HPC) techniques hold a significant advantage in this competitive arena. The emergent domain, termed HPC-Quantum, showcases instances where digital and analog methodologies complement each other for optimal performance.

This field bears tremendous excitement, aiming to harness quantum capabilities as accelerators, akin to GPUs, termed Quantum Processing Units (QPUs). It intends to strategically offloads quantum-favorable tasks to QPUs while leveraging CPUs/GPUs for other computations.

In the scope of this project, our focus pertains solely to digital quantum simulations. In the present landscape of noisy and error-prone quantum computing, digital simulations serve as indispensable tools for various reasons. Practical constraints, such as the exorbitant cost and meticulous setup requirements for real quantum hardware that has stringent environmental conditions (temperature, vacuum, laser precision, etc.), coupled with the inherent susceptibility of qubits to decoherence, highlight the importance of digital simulators. Particularly in the realm of quantum algorithm development, these simulators significantly aid researchers, ensuring efficiency for shorter test times among other benefits.

Broadly categorized, digital quantum circuit simulations primarily encompass two techniques: Statevector Simulation [9] (or Density Matrix Simulation) and Tensor Network Simulation [5].

1.1 Statevector Simulation

Statevector Simulation involves the manipulation of matrices representing quantum circuits. A quantum circuit, depicted as a sequence of matrices, culminates in a final matrix termed the Unitary Matrix, obtained through the multiplication (contraction) of matrices within the chain. Each qubit in the quantum system corresponds to a vector of dimension n for n qubits. This state evolves with the introduction of 'gates' (unitary matrices) within the chain, constituting the process of Statevector simulation.

1.1.1 Challenge: O(2^n) Memory Requirements. The matrices within a quantum circuit grow exponentially with the number of qubits. For instance, a 4-qubit quantum circuit requires a unitary matrix of 16x16, while a

5-qubit system demands a 32x32 matrix, creating memory limitations. Typically, digital statevector simulation using dense matrices encounters memory constraints, with an unofficial cutoff around 16 qubits necessitating 2^{16} times the sizeof(complex) memory.

1.1.2 Solution 1: DIAQ [4]. To mitigate this challenge, the authors developed DIAQ, an innovative sparse matrix format tailored for structured sparsity. This format optimizes memory usage, enabling simulations with a higher number of qubits within a single computational node. Figure 3 illustrates the advantageous use of this custom-built sparse format, further elaborated in section 2.5.

1.1.3 Solution 2: Tensor Networks [10]. Tensor Networks, mathematical constructs representing large tensors, offer an alternative simulation approach. By decomposing a large tensor into smaller interconnected tensors, memory explosion encountered in statevector simulations is deferred.

1.2 Tensor Network Simulation

This technique deals with higher-dimensional tensors to represent quantum circuits as graphs, with nodes denoting gate operations and edges signifying interactions. Additionally, Tensor Networks can represent Hamiltonians and contract them for observable outcomes, without explicit quantum circuit involvement.

The development of DIAQ encompasses support for both statevector and tensor network-based simulators. DIAQ adeptly manages dimensional tensors by converting them into matrices post-reshape operations, ensuring preservation of their sparse patterns.

For instance, the quantum circuit represented in Figure 1 when converted to a tensor network using QUIMB [5] looks like Figure 2.

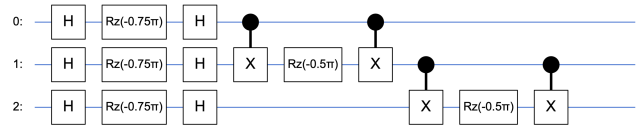


Figure 1. SupermarQ [11] HAM quantum circuit.

2 Literature Review

This section presents an in-depth exploration of various sparse matrix formats relevant to DIAQ. Additionally, it references discussions related to tensor network libraries from the previous semester's parallel systems course project [4]. Furthermore, brief discussions about SSE [2] and AVX [1] are included below.

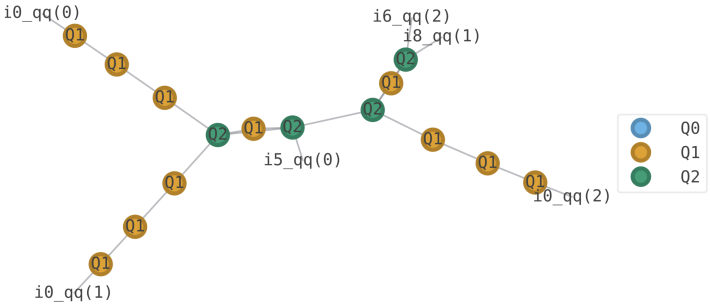


Figure 2. SupermarQ [11] HAM Tensor-Network Representation. The circles represent tensors in the network. The lines (edges) represent bond interactions. The measurements have been removed, since this is a unitary simulation. NOTE: Generated using QUIMB [5].

2.1 DIA Format

The Scipy's DIA (Diagonal) format [12] serves as a representation method for sparse matrices by arranging elements in a diagonal-major pattern, employing offsets to denote positions beyond the matrix's immediate scope. Let's examine a matrix as an example:

$$\begin{bmatrix} a & 0 & 0 & b \\ 0 & c & 0 & 0 \\ 0 & 0 & d & 0 \\ e & 0 & 0 & f \end{bmatrix}$$

In the DIA format, non-zero elements are stored with respect to their diagonals. For this matrix, the non-zero elements would be stored as:

values = [a, c, d, f, *, *, *, e, b, *, *, *]

Here, '*' denotes non-existent or empty positions within the matrix, essentially representing 'NA'. The corresponding offsets would be:

offsets = [-3, 0, 3]

These offsets signify the positions of the non-zero elements along the diagonals. Specifically, the values are aligned based on their diagonal positions relative to the main diagonal (0 offset).

This representation allows for a compact storage scheme, where non-zero values are organized in line with their respective diagonal offsets, effectively capturing the essential elements of the matrix in a structured diagonal-major format.

2.2 COO Format

The COO (Coordinate List) format [8] is another sparse matrix representation, utilizing three arrays to store non-zero elements' coordinates and values. For instance, considering the same matrix:

$$\begin{bmatrix} a & 0 & 0 & b \\ 0 & c & 0 & 0 \\ 0 & 0 & d & 0 \\ e & 0 & 0 & f \end{bmatrix}$$

In COO format, the non-zero elements would be stored as: row_indices = [0, 0, 1, 2, 3, 3], col_indices = [0, 3, 1, 2, 0, 3], values = [a, b, c, d, e, f], representing their positions and values.

2.3 SSE

SSE (Streaming SIMD Extensions) [2] is an instruction set designed to enhance CPU performance by processing multiple data elements simultaneously. For instance, it can perform operations like adding multiple pairs of single-precision floating-point numbers at once.

2.4 AVX

AVX (Advanced Vector Extensions) [1] extends SSE, offering more advanced vectorization capabilities. For example, it can handle larger vectors (8) and perform operations on more data elements simultaneously than SSE.

2.5 Why DIAQ for Quantum?

This subsection delves into the significance of using DIAQ for quantum simulations. It highlights DIAQ's memory-saving advantages and its suitability for accommodating simulations involving a larger number of qubits within a single computational node. Illustratively, Figure 3 demonstrates the memory-saving benefits of DIAQ by showcasing sparsity patterns commonly encountered in quantum simulations.

DIAQ, inspired by Scipy's DIA format, focuses specifically on quantum simulations ('Q' representing its quantum focus). Its similarity to Scipy's DIA format allows for efficient utilization in quantum simulation contexts.

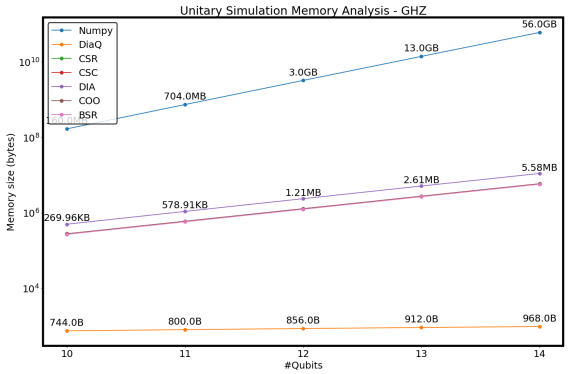


Figure 3. DIAQ Memory Savings.

3 Methodology

In this section, we will discuss the parallelization approaches incorporated into DIAQ to optimize its performance. Specifically, we will focus on three major bottlenecks identified last semester: transpose, format conversion, and tensor-dot.

3.1 AVX2 for spGEMM (Tensor-Dot)

We have upgraded from SSE to AVX2 and optimized byte alignment to better support AVX2. Both are extensions to the x86-64 instruction set for SIMD (Single Instruction, Multiple Data) instructions. This means performing the same operation on multiple variables simultaneously using a single instruction. AVX2 is a newer extension compared to SSE and features 256-bit registers instead of 128-bit, effectively doubling the workload processed in a single instruction.

3.2 GPU for Format Conversion using CUDA [7]

We have implemented a CUDA kernel to convert dense matrices to DIAQ (another significant bottleneck identified last semester). As outlined in 4, we use the GPU to convert row index and column index in the dense version to diagonal index and the position in that diagonal.

3.3 Performing spMV on GPU

We have implemented a CUDA kernel to perform spMV which in intended for later use with statevector simulations. The kernel is outlined in 5, it assumes a flattened compact version of the scipy's DIA format as input.

3.4 Transpose Operation (In-Progress)

We explored improvements to the matrix transpose, and further research is ongoing, though we haven't devised a novel tensor transpose algorithm yet. Here are our detailed thoughts:

1. **Recursive Approach Idea:** The concept involves employing a 2D matrix transpose for a 3D tensor transpose, a 3D tensor transpose for a 4D tensor transpose, and so forth, following a recursive approach.

```
__global__ void convertToDQ(
    long size, long matrixSizeX, long matrixSizeY,
    long* devDiagIdx, long* devDiagPos
) {
    // Calculate the global index of the current thread
    long idx = blockIdx.x * (blockDim.x * blockDim.y * blockDim.z)
             + threadIdx.z * (blockDim.x * blockDim.y)
             + threadIdx.y * blockDim.x + threadIdx.x;

    // Check if the current index is within the valid range
    if (idx < size) {
        // Calculate the diagonal index for the current index
        long dIdx = ((idx - matrixSizeY * (idx / matrixSizeY)) % matrixSizeY)
                  - (idx / matrixSizeY);

        long dPos;
        // Calculate the diagonal position based on the diagonal index
        if (dIdx <= 0) {
            dPos = ((idx - matrixSizeY * (idx / matrixSizeY)) % matrixSizeY);
        } else {
            dPos = ((idx - matrixSizeY * (idx / matrixSizeY)) % matrixSizeY)
                 - dIdx;
        }

        // Store the calculated diagonal index and position in the corresponding
            arrays
        devDiagIdx[idx] = dIdx;
        devDiagPos[idx] = dPos;
    }
}
```

Figure 4. CUDA Kernel: convertToDQ

Algorithm 1 Recursive Tensor Transpose

RecursiveTensorTranspose*tensor*, *axes*
transposed_tensor ← create a new tensor with dimensions according to *axes* *dims* ← dimensions of *tensor* **if** *dims* = 2 **then**
return MatrixTranspose(*tensor*, *axes*)
else
  for *slice* in *tensor* along the first axis according to *axes* **do**
    transposed_slice ← **RecursiveTensorTranspose**(*slice*, remaining axes in *axes*)
    Place *transposed_slice* into the appropriate position in *transposed_tensor*
  end for
  return *transposed_tensor*
end if=0

2. **Using Permutation Matrices:** Although effective for matrices, we have moved away from this technique due to challenges in generalizing it for higher dimensional tensors. For matrices, we already possess an O(diagonals) algorithm for transpose.

4 Experimental Setup

The experiments encompassed a comprehensive evaluation, beginning with an assessment of AVX2's performance compared to SSE on SupermarQ's benchmark circuits: Greenberger–Horne–Zeilinger (GHZ), Hamiltonian (HAM), and Mermin-Bell. Initial observations revealed no significant performance gains. To delve deeper into these results, we employed likwid to measure retired instructions and Single Precision FLOP/s across all circuits. This analysis was conducted while employing byte-aligned memory allocation to optimize AVX2 utilization.

Moreover, we investigated the format conversion performance for the SupermarQ [11] benchmark circuits, executed on both CPUs and GPUs.

All experiments were conducted on a single node equipped with an AMD EPYC 7302P 16-Core Processor operating at 3.00 GHz. This node is part of the ARC cluster [6] located in the CSC department at NC State University, boasting 192GB of DDR4 RAM and 32 logical cores.

```
__global__ void spmv_dia_kernel(long int num_rows, long int num_cols, long int
    num_diags, long int num_elements, long int *diagonal_indices, long int *
    cum_diagonal_lengths, float *real_data, float *imag_data, float *x_real_values,
    float *x_imag_values, float *y_real_values, float *y_imag_values) {

    // Calculate the index of the current row processed by the thread
    long int y_row_idx = blockDim.x * blockIdx.x + threadIdx.x;

    // Check if the current row index is within the valid range
    if(y_row_idx < num_rows) {

        // Initialize variables for the dot product calculation
        float real_dot = 0;
        float imag_dot = 0;

        // Loop through columns to compute dot product
        for(long int x_row_idx = 0; x_row_idx < num_cols; x_row_idx++) {

            // Calculate the diagonal index
            long int d_idx = x_row_idx - y_row_idx;

            // Find the corresponding diagonal number using d_idx
            long int d_num = 0;
            while(d_num < num_diags && diagonal_indices[d_num] != d_idx) {
                d_num++;
            }

            // Calculate d_pos and offset_in_data based on d_idx and d_num
            long int d_pos = (d_idx <= 0) ? x_row_idx : x_row_idx - d_idx;
            long int offset_in_data = (d_num == 0) ? d_pos : d_pos +
                cum_diagonal_lengths[d_num - 1];

            // Perform the dot product calculation for real and imaginary parts
            real_dot += real_data[offset_in_data] * x_real_values[x_row_idx] -
                imag_data[offset_in_data] * x_imag_values[x_row_idx];
            imag_dot += real_data[offset_in_data] * x_imag_values[x_row_idx] +
                imag_data[offset_in_data] * x_real_values[x_row_idx];

        }
        // Store the resulting dot products in the output arrays
        y_real_values[y_row_idx] = real_dot;
        y_imag_values[y_row_idx] = imag_dot;
    }
}
```

Figure 5. CUDA Kernel: spmv_dia_kernel

The CPU specifications comprise 32KB of L1 data cache, 32KB of L1 instruction cache, 512KB of L2 cache, and 16MB of L3 cache per CPU. Each CPU encompasses 16 cores and supports 2 threads per core, organized into 4 NUMA nodes.

Furthermore, the node is furnished with an NVIDIA GeForce RTX 4060 Ti GPU. The experiments utilized GPU driver version 535.86.10 and CUDA version 12.2 for GPU-based computations.

5 Results & Discussion

The results depicted in Figure 8 indicate that AVX2 demonstrates enhanced performance, particularly for larger lengths and with proper byte-alignment assistance during memory allocation. We conducted an analysis of the instruction count across various alignment strategies during a benchmark run of SupermarQ's GHZ for 8 qubits. Surprisingly, we observed that SSE exhibited a lower count of retired instructions, as depicted in Figure

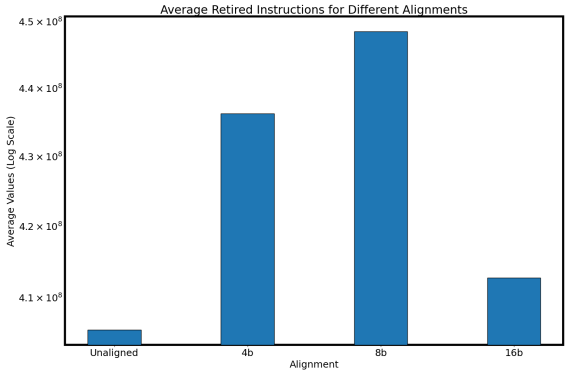


Figure 6. Number of Instruction comparison for various alignment sizes.

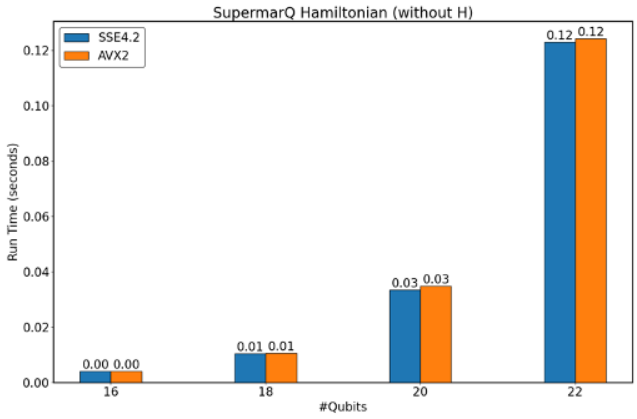


Figure 7. SSE vs AVX (HAM chain multiplication – spGEMM)

6. In response, we offer detailed considerations regarding the potential reasons behind this discrepancy:

- Despite all code being compiled at optimization level 03, it appears that optimization strategies might not solely account for the variance in retired instructions between SSE and other alignment strategies.
- Differences in the instruction sets employed by SSE and alternative alignment strategies could contribute to the observed differences in retired instructions. A more in-depth analysis is imperative to elucidate these differences.
- Variances in memory access patterns influenced by SSE and alternative alignment strategies may potentially contribute to the differences in retired instruction counts. Further investigation is essential to substantiate this hypothesis.

Performance-wise, SSE4.2 appears to outperform AVX2, achieving better average FLOP/s as observed in Figure 9. However, it is speculated that AVX2 should excel over SSE4.2 for Double Precision since AVX2 (8-vectorization) inherently supports double precision, while SSE4.2 might face challenges with 4-vectorization.

Regarding the utilization of CUDA for format conversion, Figure 10 displays no significant improvement over the CPU version. This may be attributed to the lack of density for the conversion. Authors plan to implement the DASP idea and enforce bandwidth improvements for format conversion.

While the outcomes with CUDA may not directly indicate positive results, authors believe that the tooling, scripting, and linking around the relatively new library DIAQ are now solid. This allows authors and collaborators to seamlessly implement and test new CUDA kernels.

6 Conclusion

This study aimed to enhance an ongoing numerical library for digital quantum simulations. The porting of SSE to AVX2 for spGEMM, implementation of CUDA kernels for format conversion and spMV, and extensive research on transpose algorithms are actively underway.

Tests involving chain matrix multiplication of Qiskit unitaries were conducted to evaluate the changes. Initially, AVX did not perform as well as SSE. Further investigation revealed that implementing byte-aligned memory better suited AVX2, enabling it to match SSE4.2 for all qubits and surpass it at the 14 qubit mark for the Hamiltonian benchmark quantum circuit. Additionally, the library was prepared for statevector simulation by implementing the spMV CUDA kernel.

An advantageous outcome of this project is the enhanced ease of installation and the improved generic nature of the CMake scripts and tooling quality. Additionally, Python wrappers have been developed for seamless integration with other Python libraries, as outlined in the previous semester's literature survey [4].

7 Future Work

The authors plan to undertake the following tasks in the near future:

- Enhance existing CUDA kernels or develop new ones for improved performance and functionality.
- Explore and implement an optimized algorithm for tensor transposition to enhance efficiency.
- Investigate and analyze diagonal alterations to optimize the spM-spM CUDA kernel.
- Conduct further statevector simulation experiments utilizing spMV and present the results for evaluation and analysis.

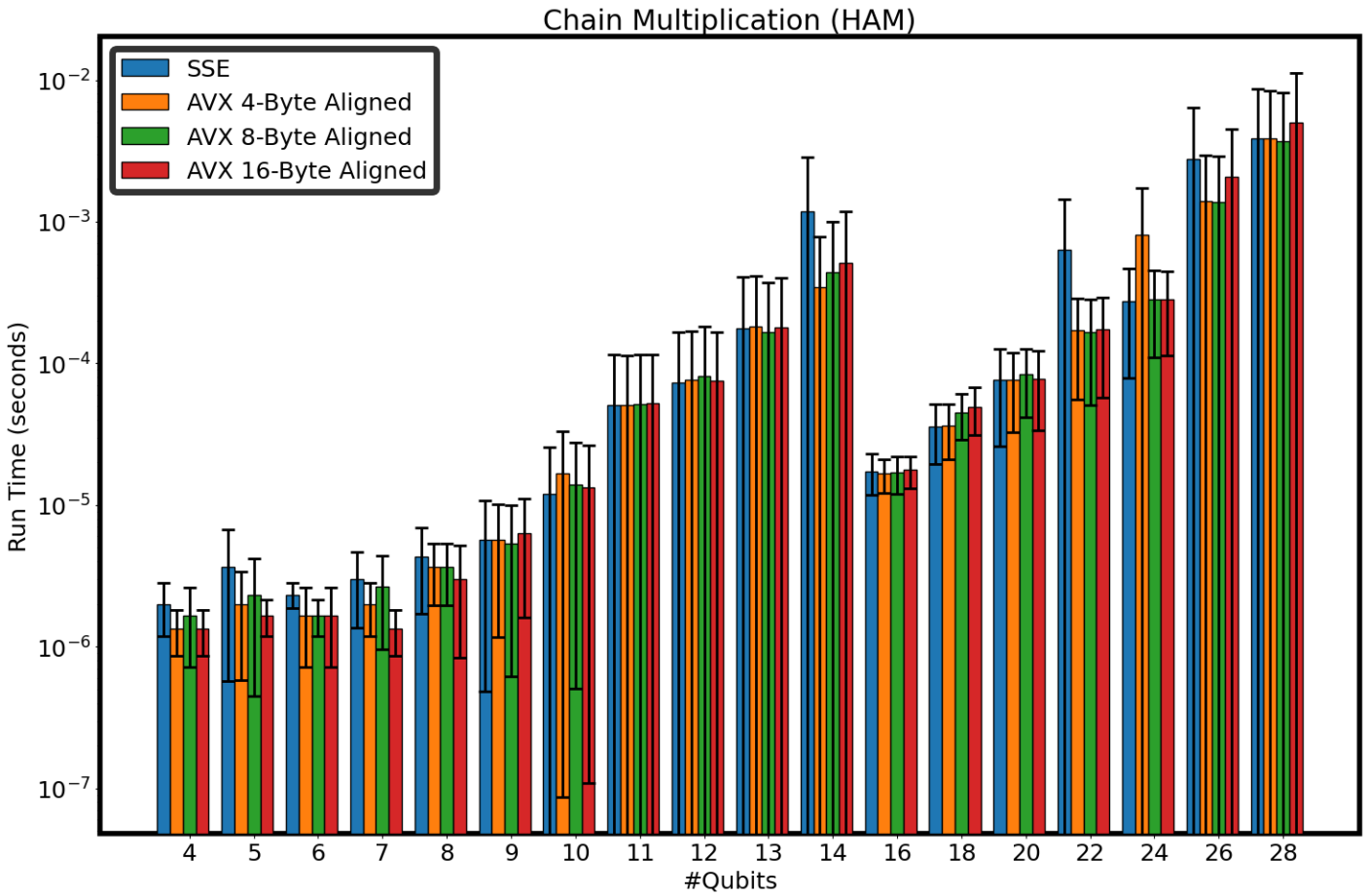


Figure 8. Chain spGEMM – Hamiltonian quantum circuit simulation

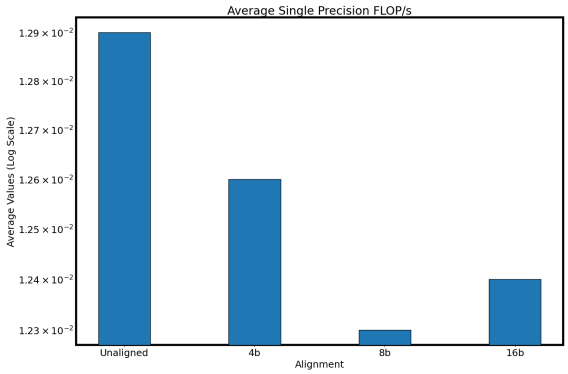


Figure 9. Single Precision FLOP/s for various alignment sizes.

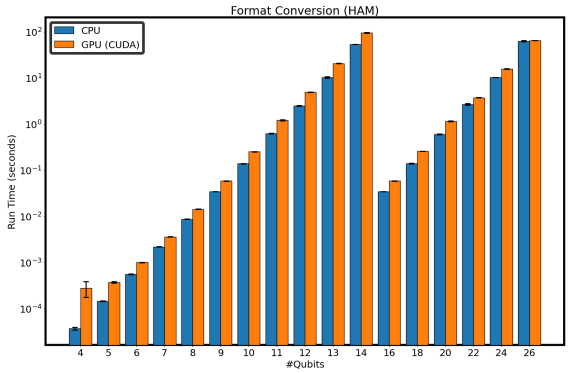


Figure 10. Format Conversion (HAM) – CPU vs GPU

Acknowledgement

I am deeply thankful to Dr. Jiajia Li for her invaluable guidance and consistent support during my course project. Her expert advice and feedback have played a pivotal role in steering this project in the right direction amidst its complexities. I truly appreciate her mentorship and expertise, which have immensely enriched my learning journey.

References

[1] Simd avx, 2018.

[2] Simd sse4.2, 2023.

[3] Frank Arute, Kunal Arya, Ryan Babbush, et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574:505–510, 2019.

[4] Srikar Chundury, Jiajia Li, and Frank Mueller. Quantum tensor-network contraction optimization. https://github.ncsu.edu/qsparse/diaq/blob/f46eddbdf7c75db18ef7235afcdb1f86d7000b8f/discussions/ParallelSystems_course_project_2.pdf, 2023. GitHub Repository: https://github.ncsu.edu/qsparse/diaq.

[5] Johnnie Gray. quimb: A python package for quantum information and many-body calculations. *Journal of Open Source Software*, 3(29):819, 2018.

[6] Dr. Frank Mueller. Arc cluster @ ncsu. https://arcb.csc.ncsu.edu/~mueller/cluster/arc/, 2023.

[7] NVIDIA, Péter Vingelmann, and Frank H.P. Fitzek. Cuda, release: 10.2.89, 2020.

[8] PyData. Sparse: A library for sparse matrix formats in python, 2023.

[9] Qiskit. Qiskit simulators. https://qiskit.org/documentation/stable/0.24/tutorials/simulators/1_aer_provider.html, 2023.

[10] TensorNetworks. Tensor networks. https://tensornetwork.org, 2023.

[11] Teague Tomesh, Pranav Gokhale, Victory Omole, Gokul Subramanian Ravi, Kaitlin N. Smith, Joshua Viszlai, Xin-Chuan Wu, Nikos Hardavellas, Margaret R. Martonosi, and Frederic T. Chong. Supermarq: A scalable quantum benchmark suite, 2022.

[12] Virtanen, Pauli and Gommers, Ralf and Oliphant, Travis E. and Haberland, Matt and Reddy, Tyler and Cournapeau, David and Burovski, Evgeni and Peterson, Pearu and Weckesser, Warren and Bright, Jonathan and van der Walt, Stéfan J. and Brett, Matthew and Wilson, Joshua and Millman, K. Jarrod and Mayorov, Nikolay and Nelson, Andrew R. J. and Jones, Eric and Kern, Robert and Larson, Eric and Carey, CJ and Polat, Ä°lhan and Feng, Yu and Moore, Eric W. and VanderPlas, Jake and Laxalde, Denis and Perktold, Josef and Cimrman, Robert and Henriksen, Ian and Quintero, E. A. and Harris, Charles R. and Archibald, Anne M. and Ribeiro, Antônio H. and Pedregosa, Fabian and van Mulbregt, Paul and SciPy 1.0 Contributors. SciPy: Open source scientific tools for python, 2020.

Appendix

- Installing DIAQ GitHub (access limited to project members): https://github.ncsu.edu/qsparse/diaq/tree/transpose-optimization-progress

```
git clone -b transpose-optimization-progress https://github.ncsu.edu/qsparse/diaq.git
cd diaq
./run_builds.sh cpu                          # CPU with AVX
./run_builds.sh gpu                          # CUDA
./run_builds.sh all                          # CPU with AVX and CUDA
./remake_python.sh cpu                       # Python wrappers using AVX
./remake_python.sh gpu                       # Python wrappers using CUDA
```

Run tests:

```
./cpu_build/bin/tests                  # CPU
./gpu_build/bin/tests                  # GPU
```

Note: Check ‘config.cpp‘ for logger debug level.

- Installing Benchmarks library

```
git clone https://github.ncsu.edu/qsparse/benchmarks.git
cd benchmarks
./run_builds.sh all                              # Install CPU and GPU versions
./cpu_build/bench supermarq ghz 9 compare        # CPU
./gpu_build/bench supermarq ham 8 diaq           # GPU
```

- Benchmarks (with raw outputs): https://github.ncsu.edu/qsparse/benchmarks/tree/main/bench/csc591
- Overleaf Doc: https://www.overleaf.com/8342325147bzgvzqzzcqtw#147e35